

MultiheadAttention#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.activation.MultiheadAttention.html>

Description:

Allows the model to jointly attend to information from different representation subspaces. This MultiheadAttention layer implements the original architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build efficient layers from building blocks in core or using higher level libraries from the PyTorch Ecosystem. Multi-Head Attention is defined as: where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$. `nn.MultiheadAttention` will use the optimized implementations of `scaled_dot_product_attention()` when possible.

Function Signature

```
class  
torch.nn.modules.activation.MultiheadAttention(embed_dim,  
num_heads, dropout=0.0, bias=True, add_bias_kv=False,  
add_zero_attn=False, kdim=None, vdim=None,  
batch_first=False, device=None, dtype=None)[source]#
```

Parameters

```
forward(query, key, value, key_padding_mask=None,  
need_weights=True, attn_mask=None,  
average_attn_weights=True, is_causal=False)[source]#
```

Parameters

Return type

Returns:

merged_mask

Code Examples

```
>>> multihead_attn = nn.MultiheadAttention(embed_dim, num_heads)  
>>> attn_output, attn_output_weights = multihead_attn(query,  
key, value)
```

Notes & Warnings

NOTE

Note batch_first argument is ignored for unbatched inputs.

Detailed Documentation

Documentation

Allows the model to jointly attend to information from different representation subspaces.

This MultiheadAttention layer implements the original architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build efficient layers from building blocks in core or using higher level libraries from the PyTorch Ecosystem.

Multi-Head Attention is defined as:

where $\text{head}_i = \text{Attention}(QW_iQ, KW_iK, VW_iV)$ $\text{head}_i = \text{Attention}(QW_iQ, KW_iK, VW_iV)$

`nn.MultiheadAttention` will use the optimized implementations of `scaled_dot_product_attention()` when possible.

In addition to support for the new `scaled_dot_product_attention()` function, for speeding up Inference, MHA will use fastpath inference with support for Nested Tensors, iff:

self attention is being computed (i.e., query, key, and value are the same tensor).

inputs are batched (3D) with `batch_first==True`

Either autograd is disabled (using `torch.inference_mode` or `torch.no_grad`) or no tensor argument requires_grad

training is disabled (using `.eval()`)

`add_zero_attn` is False

`kdim` and `vdim` are equal to `embed_dim`

if a `NestedTensor` is passed, neither `key_padding_mask` nor `attn_mask` is passed

If the optimized inference fastpath implementation is in use, a `NestedTensor` can be passed for query/key/value to represent padding more efficiently than using a padding mask. In this case, a `NestedTensor` will be returned, and an additional speedup proportional to the fraction of the input that is padding can be expected.

`embed_dim` – Total dimension of the model.

`num_heads` – Number of parallel attention heads. Note that `embed_dim` will be split across `num_heads` (i.e. each head will have dimension `embed_dim // num_heads`).

`dropout` – Dropout probability on `attn_output_weights`. Default: 0.0 (no dropout).

`bias` – If specified, adds bias to input / output projection layers. Default: True.

`add_bias_kv` – If specified, adds bias to the key and value sequences at `dim=0`. Default:

False.

`add_zero_attn` – If specified, adds a new batch of zeros to the key and value sequences at `dim=1`. Default: False.

`kdim` – Total number of features for keys. Default: None (uses `kdim=embed_dim`).

`vdim` – Total number of features for values. Default: None (uses `vdim=embed_dim`).

`batch_first` – If True, then the input and output tensors are provided as (batch, seq, feature). Default: False (seq, batch, feature).

Compute attention outputs using query, key, and value embeddings.

Supports optional parameters for padding, masks and attention weights.

`query` (Tensor) – Query embeddings of shape (L, E_q) for unbatched input, (L, N, E_q) when `batch_first=False` or (N, L, E_q) when `batch_first=True`, where L is the target sequence length, N is the batch size, and E_q is the query embedding dimension `embed_dim`. Queries are compared against key-value pairs to produce the output. See “Attention Is All You Need” for more details.

`key` (Tensor) – Key embeddings of shape (S, E_k) for unbatched input, (S, N, E_k) when `batch_first=False` or (N, S, E_k) when `batch_first=True`, where S is the source sequence length, N is the batch size, and E_k is the key embedding dimension `kdim`. See “Attention Is All You Need” for more details.

`value` (Tensor) – Value embeddings of shape (S, E_v) for unbatched input, (S, N, E_v) when `batch_first=False` or (N, S, E_v) when `batch_first=True`, where S is the source sequence length, N is the batch size, and

`EvE_vEv` is the value embedding dimension `vdim`. See “Attention Is All You Need” for more details.

`key_padding_mask` (Optional[`Tensor`]) – If specified, a mask of shape $(N,S)(N, S)(N,S)$ indicating which elements within key to ignore for the purpose of attention (i.e. treat as “padding”). For unbatched query, shape should be $(S)(S)(S)$. Binary and float masks are supported. For a binary mask, a `True` value indicates that the corresponding key value will be ignored for the purpose of attention. For a float mask, it will be directly added to the corresponding key value.

`need_weights` (bool) – If specified, returns `attn_output_weights` in addition to `attn_outputs`. Set `need_weights=False` to use the optimized `scaled_dot_product_attention` and achieve the best performance for MHA. Default: `True`.

`attn_mask` (Optional[`Tensor`]) – If specified, a 2D or 3D mask preventing attention to certain positions. Must be of shape $(L,S)(L, S)(L,S)$ or $(N\cdot\text{num_heads},L,S)$ $(N\cdot\text{num_heads}, L, S)(N\cdot\text{num_heads},L,S)$, where NNN is the batch size, LLL is the target sequence length, and SSS is the source sequence length. A 2D mask will be broadcasted across the batch while a 3D mask allows for a different mask for each entry in the batch. Binary and float masks are supported. For a binary mask, a `True` value indicates that the corresponding position is not allowed to attend. For a float mask, the mask values will be added to the attention weight. If both `attn_mask` and `key_padding_mask` are supplied, their types should match.

`average_attn_weights` (bool) – If `true`, indicates that the returned `attn_weights` should be averaged across heads. Otherwise, `attn_weights` are provided separately per head. Note that this flag only has an effect when `need_weights=True`. Default: `True` (i.e. average weights across heads)

`is_causal` (bool) – If specified, applies a causal mask as attention mask. Default: `False`.

Warning: `is_causal` provides a hint that `attn_mask` is the causal mask. Providing incorrect hints can result in incorrect execution, including forward and backward compatibility.

`tuple[torch.Tensor, Optional[torch.Tensor]]`

`attn_output` - Attention outputs of shape (L, E) when input is unbatched, (L, N, E) when `batch_first=False` or (N, L, E) when `batch_first=True`, where LLL is the target sequence length, NNN is the batch size, and EEE is the embedding dimension `embed_dim`.

`attn_output_weights` - Only returned when `need_weights=True`. If `average_attn_weights=True`, returns attention weights averaged across heads of shape (L, S) when input is unbatched or (N, L, S) , where NNN is the batch size, LLL is the target sequence length, and SSS is the source sequence length. If `average_attn_weights=False`, returns attention weights per head of shape $(\text{num_heads}, L, S)$ when input is unbatched or $(N, \text{num_heads}, L, S)$.

`batch_first` argument is ignored for unbatched inputs.

Determine mask type and combine masks if necessary.

If only one mask is provided, that mask and the corresponding mask type will be returned. If both masks are provided, they will be both expanded to shape $(\text{batch_size}, \text{num_heads}, \text{seq_len}, \text{seq_len})$, combined with logical or and mask type 2 will be returned

- :param `attn_mask`: attention mask of shape $(\text{seq_len}, \text{seq_len})$, mask type 0
- :param `key_padding_mask`: padding mask of shape $(\text{batch_size}, \text{seq_len})$, mask type 1
- :param `query`: query embeddings of shape $(\text{batch_size}, \text{seq_len}, \text{embed_dim})$

merged mask `mask_type`: merged mask type (0, 1, or 2)

